Commute Optimizer — Implementation and Experimental Report

1. Objective and experimental setup

This project optimizes a complete commute configuration $x = \langle P, t_{start} \rangle$: a directed, loop-free route and a discrete departure time. Travel time is evaluated by the supplied FIFO simulator, which advances the clock edge by edge and applies the Gaussian congestion multiplier at each edge's actual entry time.

The benchmark in this report uses the supplied `config.json` and the default `toy_graph.json` / `rush_hour.json` pair:

- Origin / destination: `Home` / `Uni`
- Departure window: 465–525 minutes after midnight (07:45–08:45), step 5 minutes
- Hill Climbing: `max_sideways=100`, `max_restarts=10`
- Simulated Annealing: `T0=120`, `alpha=0.992`, `Tmin=0.001`, `max_iterations=3000`
- Local Beam Search: `K=8`, `max_iterations=150`
- Genetic Algorithm: population 50, 120 generations, crossover 0.85, mutation 0.25, elitism 3, tournament size 3

Runtime figures are means over five runs with seeds 0–4, measured using `time.perf_counter()` in the current execution environment. Best route and objective are the best result observed across those five runs. The convergence figure is a single reproducible run with seed 42.

2. Operator design and route validity

Neighborhood generation

`get_neighbors` produces two classes of moves:

1. **Departure-time mutation:** shift departure by one legal time step, either $+\Delta t$ or $-\Delta t$.
2. **Local route detours:** choose a contiguous section of the current route and replace it with a short alternative directed path between the same boundary nodes. The detour search uses depth-first enumeration with a bounded length and excludes nodes already used by the unaffected prefix or suffix.

Each candidate is passed through `CommuteProblem.is_valid_state`. This checks the origin and destination, directed-edge continuity, absence of repeated nodes, and the departure-time bounds. A set removes duplicate configurations.

Crossover

`crossover` finds shared intermediate nodes and attempts to splice the prefix of parent 1 with the suffix of parent 2 at a common junction. Every proposed child

is validated before it is returned. If the parents have no shared intermediate node, or every splice is invalid, the operator keeps the lower-cost parent's route and uses the nearest legal grid point to the parents' mean departure time. This fallback avoids disconnected or cyclic children.

3. Algorithm implementations

- **Hill Climbing:** evaluates the neighborhood and takes a lowest-cost improving state. Equal-cost sideways moves are allowed up to `max_sideways`; a per-restart visited set prevents cycling. Up to `max_restarts` randomized valid configurations explore other basins.
- **Simulated Annealing:** samples one neighbor at each iteration. Improvements are accepted; a worse move is accepted with probability $\exp(-\Delta E/T)$. Temperature follows $T_{k+1} = \alpha T_k$.
- **Local Beam Search:** holds up to K configurations, pools successors from all beam members, and selects the best K collectively. The selection first favors distinct route structures, then fills remaining slots with the next-best distinct states. This reduces immediate population collapse.
- **Genetic Algorithm:** initializes a population of valid states. Tournament selection maximizes fitness $1/(1 + f(x))$, crossover combines parent routes at shared junctions, mutation selects a random valid neighbor, and elitism copies the best individuals unchanged to the next generation.

All solvers keep the best objective value seen in `self.history`; `iterations` records the number of solver steps executed.

4. Benchmark results

The complete toy instance has 12 simple directed routes and 13 legal departure times, or 156 complete configurations. Enumerating those configurations gives an exact optimum of 41.7726 minutes, so the best value reported below matches the exhaustive baseline for this small benchmark.

Algorithm	Best travel time \(f(x^*)\)	Departure	Mean runtime (s)	Mean iterations
Hill Climbing	41.7726 min	08:45	0.0203	48.8
Simulated Annealing	41.7726 min	08:45	0.1728	1457
Local Beam Search	41.7726 min	08:45	0.0043	2.2
Genetic Algorithm	41.7726 min	08:45	0.2866	120

Best route returned by all four algorithms: `Home → B1 → B2 → H2 → Uni`.

The best result is the same across all four methods in this small graph. Beam Search used the fewest iterations on average. Hill Climbing had the lowest mean runtime. Simulated Annealing spent more iterations exploring and cooling, while the Genetic Algorithm ran the configured 120 generations. The number of iterations is not directly comparable as an equal unit of work: one beam iteration evaluates successors from several states, and one GA generation evaluates many population members.

5. Convergence comparison

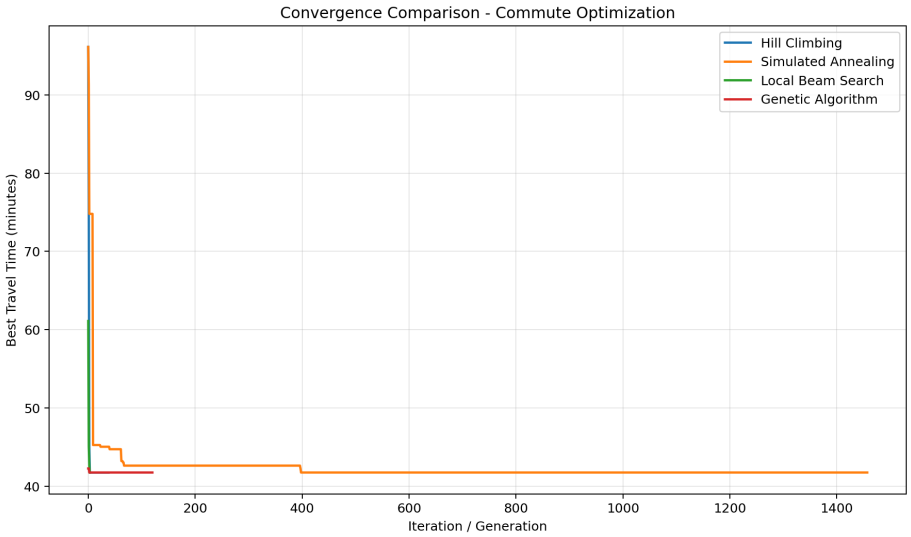


Figure 1: Convergence comparison

The plotted curves show the best-so-far objective. Hill Climbing drops quickly after inspecting local detours and time shifts. Beam Search also improves rapidly because it pools candidates from several states. Simulated Annealing shows more gradual progress and continues for many iterations while its temperature cools. The Genetic Algorithm's best-so-far curve is monotone because elitism preserves the best individuals, although the population continues evolving after the best value is found.

6. Hyperparameter sensitivity

The following probes use seed 42 and change one parameter at a time on the default toy instance. They are observations for this dataset, not universal guarantees.

- **Hill Climbing — `max_sideways` = 0, 20, 100:** all three settings reached 41.7726 minutes in 49 iterations. Sideways moves were not needed on this default map, which has no long cost-neutral plateau.
- **Simulated Annealing — `cooling_rate` = 0.5, 0.88, 0.992, 0.999:** 0.5 stopped after 17 steps at 45.3902 minutes; 0.88 after 92 steps at 45.0057 minutes; 0.992 after 1457 steps at 41.7726 minutes; 0.999 reached the 3000-step cap and also found 41.7726 minutes.
- **Local Beam Search — `beam_width` = 3, 8, 15:** all found 41.7726 minutes. Iterations were 3, 2, and 1 respectively; wider beams increase the work per iteration.
- **Genetic Algorithm — `mutation_rate` = 0.05, 0.25, 0.5:** all found 41.7726 minutes in 120 generations. Runtime increased as more mutations triggered additional neighbor generation. On harder instances, excessive mutation can also disrupt useful building blocks.

These are one-seed sensitivity probes (seed 42), not universal claims. Results can differ on larger maps and other traffic scenarios.

7. Analytical questions

Q1 — Continuous Optimization and Gradient Descent

Classical Gradient Descent is not suitable for the route component P. A route is a discrete sequence of node identifiers, not a point in a differentiable vector space. Small numerical changes to a node identifier have no meaningful interpretation, and arbitrary vector updates would not preserve directed-edge connectivity or the no-cycle constraint.

With P fixed, a continuous relaxation of departure time can be optimized by gradient methods if the objective is differentiable on the time interval. A sufficient practical condition is that every edge congestion function $C_e(t)$ is continuously differentiable and finite, so the recursively computed arrival times and total travel time are differentiable by the chain rule. The departure-time bounds must be handled with a projected or constrained method. In the actual assignment, departure time is on a discrete grid, so a continuous optimum must be rounded or the grid itself searched. For the FIFO property, the edge arrival-time map $A_e(t) = t + \tau(e, t)$ must be nondecreasing. Since $\tau(e, t) = t_e^{base}(1 + C_e(t))$, the differentiable condition is:

$$1 + t_e^{base} C_e'(t) \geq 0.$$

Gaussian congestion waves are smooth, but the resulting objective can still be non-convex and may have multiple local minima.

Q2 — Local Search versus Incremental Graph Search

The complete-state representation directly evaluates the combination of a whole route and a departure time. This matters because changing the departure time changes the entry time into every downstream edge, and therefore changes the cost of the entire route. Route topology and departure time are coupled; a local-search operator can modify either part while the evaluator scores the complete plan.

Time-dependent Dijkstra is not inherently incorrect: under FIFO edge arrival functions and suitable time-dependent edge evaluation, label-setting Dijkstra can remain valid. Incremental methods become more difficult when FIFO is violated, waiting is allowed or strategically useful, time windows or turn-dependent constraints enlarge the state, or several resources/objectives must be tracked. In those cases, a node alone may not summarize enough information and a time-expanded or multi-label search may be required. Local search handles the full black-box objective more flexibly, at the cost of losing a general optimality guarantee.

Q3 — Local Beam Search versus Genetic Algorithm

The key advantage of GA crossover is **recombination**: useful route segments from two different parents can be combined at a common junction. Local Beam Search only selects from the pooled set of current successors; it does not directly combine partial solutions from separate candidates. Crossover can therefore create a route that neither parent nor a one-step beam successor contains.

Beam collapse occurs when several parents share the same attractive basin and their best successors are nearly identical. The implementation reduces this by favoring distinct route structures. Other remedies include novelty penalties, minimum route-distance thresholds, random immigrants, stochastic selection, and occasional mutations. The GA maintains diversity through random initialization, tournament selection, crossover, and mutation, although excessive selection pressure or low mutation can still cause premature convergence.

Q4 — Plateaus and Sideways Moves

A plateau can occur when several urban detours have equal free-flow time and experience the same or nearly uniform congestion. A grid with equal-length Manhattan routes is a concrete example: changing one junction may leave total travel time unchanged even though it moves the route toward a better perimeter corridor. A shoulder can also appear when departure-time changes are small relative to a broad, flat traffic wave.

Increasing `max_sideways` gives Hill Climbing more opportunities to cross such neutral regions, but increases evaluation cost and can cause wandering. A visited-state set and a finite sideways budget are necessary to avoid cycling. On the supplied default toy graph, testing 0, 20, and 100 sideways moves pro-

duced the same 41.7726-minute result and 49 iterations; thus this instance does not demonstrate a material quality benefit from sideways moves.

Q5 — Cooling Schedule Dynamics

If T_0 is too high, the algorithm accepts many worse moves and behaves almost randomly for too long. If it is too low, it behaves like greedy Hill Climbing and may become trapped early. The cooling rate controls how long exploration remains active. In the seed-42 probe, $\alpha = 0.5$ (rapid quenching) stopped after 17 steps at 45.3902 minutes, whereas $\alpha = 0.992$ ran 1457 steps and reached 41.7726 minutes. With $\alpha = 0.999$, the 3000-iteration cap was reached; it also found the best value but consumed more runtime. These observations illustrate the exploration/runtime trade-off.

Q6 — Crossover Disjointness and Epistasis

When two parents share no intermediate node, there is no safe junction at which to splice their routes. The implementation detects this case and returns the lower-cost parent's valid route with a legal blended departure time. It therefore avoids fabricating a disconnected path. The drawback is that this fallback does not create new topology; mutation or other parents must supply diversity.

Even when both parents are individually strong, their combination can be poor because route segments are context-dependent. A prefix determines the time at which the vehicle reaches the crossover junction, and that arrival time may place the suffix inside a downstream congestion peak. The joined path can be topologically valid while its total travel time is much worse. This dependence of segment quality on the surrounding route and arrival time is epistasis.

Q7 — Combinatorial State-Space Bound

Let N_t be the number of legal departure-time values, b the average out-degree, and L the number of route hops. A rough upper bound for paths of exactly L hops is:

$$|\mathcal{X}| \leq N_t b^L.$$

If paths may contain between 1 and L hops, a corresponding bound is:

$$|\mathcal{X}| \leq N_t \sum_{\ell=1}^{L} b^\ell = N_t \frac{b^{L+1} - b}{b - 1} \quad (b \neq 1).$$

The no-cycle rule makes the actual number smaller, and $L \leq |V| - 1$ for a simple path. Nevertheless, the number of route candidates grows exponentially with path length, while the time dimension multiplies it by N_t. Exhaustive

enumeration therefore becomes impractical on large networks, motivating local-search heuristics that explore only selected neighborhoods or populations.

8. Execution and reproduction

The five requested implementation modules retain their original function and class interfaces. Their `TODO` markers remain in place to identify the completed implementation points. The implementation files contain no explanatory code comments.

Run all four algorithms and save the convergence and route figures:

```
python main.py --algo all --seed 42 --plot-metrics --save-figs
```

Reproduce the five-seed runtime summary used for the comparison table:

```
python benchmark.py
```

The benchmark writes `benchmark_results.csv`. The included `convergence_comparison.png` was generated by the all-algorithm run with seed 42. The included `core/`, `visualization/`, and default toy map/scenario are runnable support copies for this standalone bundle. When applying the work to the course repository, replace only the five requested files inside `algorithms/` and the updated `config.json`; preserve the repository's original `core/`, `visualization/`, and data files.